

D7-api-service

Designing APIs That Other Developers Can Actually Use

LEARNING JOURNEY

Client ↓ HTTP Request ↓ REST API ↓ Controller ↓ Service ↓ Repository ↓ Database ↓ HTTP Response ↓ Client

KEY QUESTION

How do we design an API that is simple, predictable, reusable and easy for another developer to consume?

What Is an API?

API = Application Programming Interface

An API provides a defined way for one application to communicate with another.

Example

A frontend application wants student information.

Frontend → GET /students/101 → Student Management API → Database → JSON Response

API hides internal implementation

The client does not need to know:

- Which Java classes are used
- Which database is used
- How the query is executed
- Where the application is deployed

The client only needs to understand:

Endpoint + HTTP method + request + response

Real-World API Example

Think About an API Like a Restaurant

Customer → Menu → Order → Kitchen → Food

API equivalent:

Client → API documentation → HTTP Request → Business Logic → Database → HTTP Response

Mapping

Restaurant	API
Menu	API documentation
Order	HTTP request
Kitchen	Business logic
Food	Response
Table number	Request information

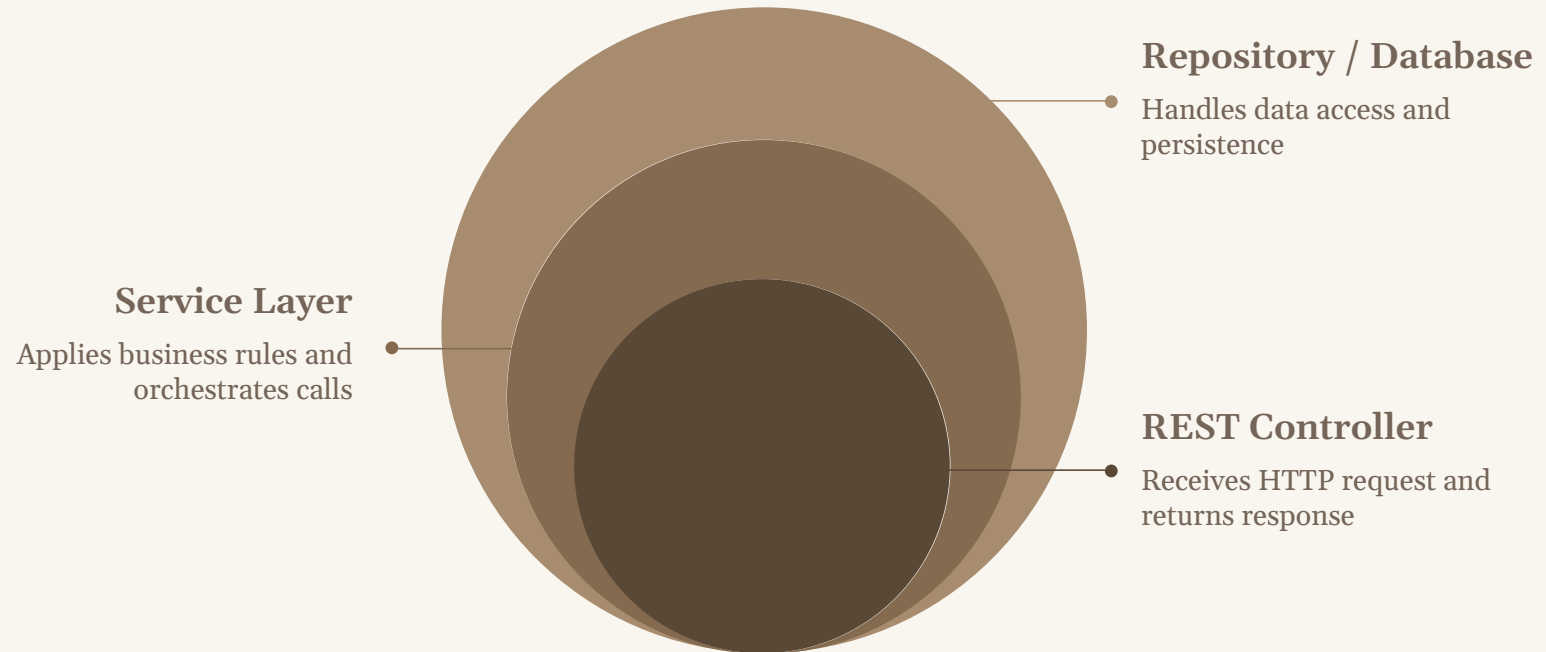
Key idea

The API is the contract between the client and server.

The Complete Request Flow

Student Management API

Browser / Frontend / Postman | | HTTP Request ↓ REST Controller | ↓ Service | ↓ Repository | ↓ Database | ↓ Repository | ↓ Service | ↓ REST Controller | | HTTP Response ↓ Client



Important

Each layer has a responsibility.

Do not put everything inside the controller.

What Happens When We Call an API?

Example

GET /students/101

Step-by-step

01

Client creates HTTP request.

02

Request reaches Spring Boot application.

03

Spring identifies the matching controller method.

04

Controller receives id = 101.

05

Service performs business logic.

06

Repository retrieves data.

07

Database returns the student.

08

Java object is converted into JSON.

09

Spring sends HTTP response.

10

Client receives the response.

REST API and Resources

REST Thinks in Terms of Resources

A resource is something the API manages.

Student Management Resources

students

courses

teachers

departments

Instead of thinking:

"What operation am I performing?"

Think:

"What resource am I working with?"

Student resource

/students

Individual student:

/students/101

Bad REST Resource Design

Avoid Action-Based URLs

Bad

```
GET /getAllStudents
GET /getStudentById/101
POST /createStudent
PUT /updateStudent/101
DELETE /deleteStudent/101
```

Why is this bad?

The URL contains the operation.

The HTTP method should communicate the operation.

Better REST API Design

Use Resource-Based URLs

```
GET /students
GET /students/101
POST /students
PUT /students/101
PATCH /students/101
DELETE /students/101
```

The URL identifies the resource.

The HTTP method identifies the action.

URL	Method
/students	GET
/students	POST
/students/101	GET
/students/101	PUT
/students/101	PATCH
/students/101	DELETE

This is one of the most important REST concepts.

HTTP Methods

CRUD Mapping

HTTP Method	Typical Purpose	CRUD
GET	Read	Read
POST	Create	Create
PUT	Replace	Update
PATCH	Partially update	Update
DELETE	Remove	Delete

Student API

GET /students
GET /students/101
POST /students
PUT /students/101
PATCH /students/101
DELETE /students/101

GET: Reading Data

Get All Students

GET /students

Response:

```
[
  { "id": 101, "name": "Rahul", "course": "Spring Boot" },
  { "id": 102, "name": "Anita", "course": "Java" }
]
```

Get One Student

GET /students/101

Response:

```
{
  "id": 101,
  "name": "Rahul",
  "course": "Spring Boot"
}
```

Spring Boot

```
@GetMapping("/students")
@GetMapping("/students/{id}")
```

POST: Creating a Resource

Request

```
POST /students
Content-Type: application/json
```

Request body:

```
{
  "name": "Rahul",
  "email": "rahul@example.com",
  "course": "Spring Boot"
}
```

Response

```
{
  "id": 101,
  "name": "Rahul",
  "email": "rahul@example.com",
  "course": "Spring Boot"
}
```

Important

The client does not normally provide the database-generated ID.

The server creates it.

PUT vs PATCH

PUT

Used to replace the resource representation.

```
PUT /students/101
```

```
{  
  "name": "Rahul",  
  "email": "rahul@example.com",  
  "course": "Java"  
}
```

Think:

"Here is the new complete representation."

PATCH

Used for partial modification.

```
PATCH /students/101
```

```
{  
  "course": "Spring Boot"  
}
```

Think:

"Change only this part."

DELETE

Delete a Student

```
DELETE /students/101
```

The server identifies student 101 and removes it.

Possible response:

```
204 No Content
```

Important

A successful DELETE does not always need to return a JSON body.

URI / Resource Naming Rules

Good URI Design

Use nouns.

```
/students  
/courses  
/teachers
```

Use plural resource names consistently.

```
/students  
/students/101
```

Avoid

```
/getStudents  
/createStudent  
/deleteStudent  
/studentAPI
```

Recommended hierarchy

```
/courses/10/students
```

Meaning:

Students belonging to course 10.

Path Variables

Identify a Specific Resource

```
GET /students/101
```

Here:

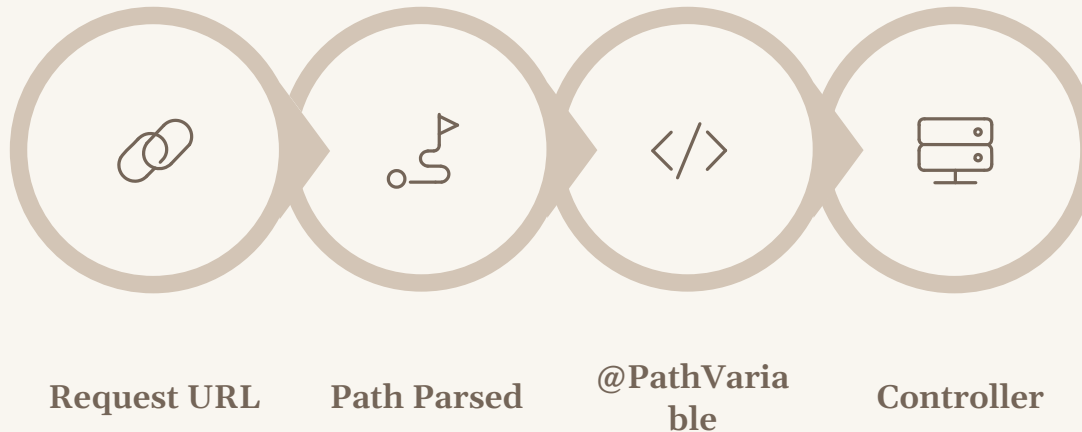
101

is part of the URL.

Spring Boot

```
@GetMapping("/students/{id}")  
public Student getStudent(@PathVariable Long id) {  
    return studentService.getStudent(id);  
}
```

Flow



Query Parameters

Used for Filtering, Searching, Sorting and Pagination

Filtering

```
GET /students?course=Java
```

Sorting

```
GET /students?sort=name
```

Pagination

```
GET /students?page=0&size=20
```

Multiple parameters

```
GET /students?course=Java&sort=name&page=0&size=20
```

Key difference

Path variable:

```
/students/101
```

Identifies a resource.

Query parameter:

```
/students?course=Java
```

Modifies how resources are retrieved.

Request Body

Data Sent by the Client

For POST:

```
{  
  "name": "Rahul",  
  "email": "rahul@example.com",  
  "course": "Spring Boot"  
}
```

Spring Boot

```
@PostMapping("/students")  
public Student createStudent(  
    @RequestBody Student student) {  
    return studentService.createStudent(student);  
}
```

What does @RequestBody do?

It tells Spring:

Convert the JSON request body into a Java object.

Response Body

Server Sends Data Back as JSON

Java object:

```
Student student = new Student(  
    101,  
    "Rahul",  
    "rahul@example.com",  
    "Spring Boot"  
);
```

Spring Boot converts it into:

```
{  
  "id": 101,  
  "name": "Rahul",  
  "email": "rahul@example.com",  
  "course": "Spring Boot"  
}
```

Key concept

Java Object ↓ Jackson ↓ JSON

And in the opposite direction:

JSON ↓ Jackson ↓ Java Object

HTTP Headers

Headers Carry Metadata

Example:

```
Content-Type: application/json
Accept: application/json
Authorization: Bearer <token>
```

Content-Type

Tells the server:

What format is being sent?

```
Content-Type: application/json
```

Accept

Tells the server:

What response format does the client want?

```
Accept: application/json
```

Content-Type vs Accept

Easy Way to Remember

Content-Type

What am I sending?

```
Content-Type: application/json
```

Accept

What do I want back?

```
Accept: application/json
```

Example

```
POST /students
Content-Type: application/json
Accept: application/json
```

Body:

```
{
  "name": "Rahul",
  "course": "Spring Boot"
}
```